Week 4 Assignment: Implementing a Creational Pattern: Factory

Master of Science

Information Technology

Object-Oriented Mthd & Pgm II

Gabriel Twizerimana

University of Denver College of Professional Studies

April 26, 2026

Faculty: Nathan Braun, MS

Dean: Bobbie Kite, PhD

Table of Contents

Design and Implementation of a Factory-Driven Parking Management System 1
Architectural Design: The Strategy and Factory Patterns 1
Challenges Encountered 2
Problem-Solving and Refinement 2
Solving the Context and Casting Issues 2
Implementing Dynamic Unit Tests 3
Standardizing the Factory Input 3
Conclusion 4
References 5

Design and Implementation of a Factory-Driven Parking Management System

The design of the parking management system centers on a modular, extensible architecture that leverages the **Strategy** and **Factory** design patterns to handle complex business logic. The primary goal was to create a system where parking fee calculations could vary dynamically based on vehicle types, calendar dates, or promotional discounts without requiring modifications to the core transaction processing logic.

Architectural Design: The Strategy and Factory Patterns

At the heart of the design is the ParkingChargeStrategy interface. This interface defines a single contract: calculateFee(ParkingPermit permit). By defining this interface, the system treats all pricing logic as interchangeable components. The design includes several concrete implementations:

- **TypeBasedStrategy**: Calculates fees based strictly on the CarType (e.g., Compact, SUV, Truck).
- **WeekdayWeekendStrategy**: Introduces calendar awareness, applying different rates based on whether the current date falls on a weekday or a weekend.
- **WeekendDiscountStrategy**: A more specialized version that applies a percentage-based discount to standard rates during the weekend.

To manage these strategies, I implemented the **ParkingChargeStrategyFactory**. This class serves as the "Information Expert" for object creation. Instead of the TransactionManager or Main class using the new keyword to instantiate specific strategies, which would create tight coupling, they simply pass a string identifier (e.g., "WEEKEND_DISCOUNT") to the factory. The factory then handles the instantiation logic and returns a generic ParkingChargeStrategy object. This ensures that the high-level components of the system remain decoupled from the low-level implementation details of specific pricing rules.

Challenges Encountered

During the implementation phase, several technical challenges surfaced, primarily regarding Java's type system and the management of static versus instance contexts.

- **The Static vs. Instance Context Conflict** A recurring challenge was the "non-static method cannot be referenced from a static context" error. This occurred when attempting to call getStrategy() directly from the main() method. Because main is static, it exists at the class level, whereas the factory methods were instance-level.
- **ClassCastExceptions and Identity Mismatches** The most significant hurdle was a java.lang.ClassCastException. This happened within the TransactionManager when the code attempted to cast the ParkingChargeStrategyFactory instance directly into a ParkingChargeStrategy. This error highlighted a conceptual misunderstanding in the initial code structure: the factory is the *producer* of the strategy, not the strategy itself.
- **Temporal Testing Logic** Testing the WeekdayWeekendStrategy presented a unique challenge because it relies on LocalDate.now(). Since the output of the code changes depending on the day it is run, a standard hard-coded unit test would fail if written on a Friday and run on a Saturday.

Problem-Solving and Refinement

To overcome these challenges, I adopted a systematic approach to debugging and refactoring.

Solving the Context and Casting Issues

To resolve the static context error, I ensured that the ParkingChargeStrategyFactory was properly instantiated within the main method. By creating a new factory object, the system could then access its instance methods.

To solve the ClassCastException, I refactored the TransactionManager.park() method. Instead of attempting a direct cast, I ensured the manager held a reference to the factory and explicitly called the getStrategy() method. This reinforced the "Program to an Interface" principle; the manager now correctly receives a ParkingChargeStrategy object from the factory and interacts with it through the interface's methods, preserving the integrity of the design pattern.

Implementing Dynamic Unit Tests

To solve the issue of time-sensitive testing, I refactored the JUnit tests to be "environmentally aware." Instead of asserting a fixed number, the tests now calculate what the expected value *should* be based on LocalDate.now() before comparing it to the strategy's output. This ensures the test suite remains green regardless of when the building is executed.

Standardizing the Factory Input

To make the factory more robust and user-friendly, I implemented input normalization. By converting the input string to uppercase and replacing spaces with underscores (e.g., "weekend discount" becoming "WEEKEND_DISCOUNT"), I reduced the likelihood of "Method Not Found" or null errors caused by minor string formatting issues. I also implemented a default case in the factory's switch statement to return a TypeBasedStrategy as a fallback, ensuring the system never crashes due to an unrecognized strategy request.

Conclusion

The resulting design is a highly flexible parking management system that demonstrates the power of creational and behavioral design patterns. By isolating the pricing logic into strategies and centralizing object creation within a factory, the system is prepared for future expansion, such as adding hourly rates or holiday surcharges—with minimal disruption to the existing codebase. The process of troubleshooting casting errors and static context issues ultimately led to a deeper understanding of object-oriented principles and a more resilient final product.

References

Martin, Robert C. 2003. *Agile Software Development, Principles, Patterns, and Practices*. Upper Saddle River, NJ: Prentice Hall.

Freeman, Eric, Elisabeth Robson, Bert Bates, and Kathy Sierra. 2020. *Head First Design Patterns*. 2nd ed. Sebastopol, CA: O'Reilly Media

Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

Shvets, Alexander. 2018. *Dive Into Design Patterns*. v2018.2.13. Refactoring.Guru

Oracle. 2024. "ClassCastException (Java SE 17 & JDK 17)." Oracle Help Center. Accessed April 26, 2026. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/ClassCastException.html